Indian Traffic Sign Classifier

Using YOLOv8n-cls Fine-Tuning

SMAI Assignment 3 - Topic T5.1: Road Safety & Traffic Vision

Shubham Paliwal (2025201009) - shubham.paliwal@students.iiit.ac.in
Rohit Rohan Sabat (2025202035) - rohit.sabat@students.iiit.ac.in
Peeyush Prashant (2025201102) - peeyush.prashant@students.iiit.ac.in
International Institute of Information Technology, Hyderabad
GitHub: https://github.com/rohitrohon/SMAI-Assignment-3

Abstract

Traffic sign recognition is a critical component of intelligent transportation systems and autonomous driving pipelines. In this project, we build an end-to-end Indian traffic sign classification system capable of recognising 84 categories of Indian road signs from photographs or short video clips. We fine-tune YOLOv8n-cls, a lightweight classification backbone with roughly 3 million parameters, on a curated Kaggle dataset of Indian traffic signs. The trained model achieves a Top-1 accuracy of 93.98% and a Top-5 accuracy of 99.62% on the held-out test set. The model is deployed through an interactive Streamlit web application supporting both single-image inference and frame-by-frame video analysis. Our system delivers strong accuracy while remaining efficient enough for real-time deployment on consumer hardware.

1 Introduction

India has one of the highest road traffic fatality rates globally, with over 150,000 deaths annually. A significant contributing factor is the lack of awareness and recognition of traffic signs by drivers, particularly in rural and semi-urban areas. Automated traffic sign recognition (TSR) can play a vital role in advanced driver-assistance systems (ADAS), autonomous vehicles, and road-safety auditing tools.

While considerable research exists on European and Chinese traffic sign datasets (e.g., GTSRB), Indian traffic signs remain underrepresented in the literature. Indian signs follow distinct design conventions prescribed by the Indian Road Congress (IRC), featuring Hindi text, unique iconography, and varied weathering conditions that pose additional challenges for automated recognition.

In this work, we address Topic T5.1 of the SMAI Assignment 3 catalogue. Our contributions are:

- Fine-tuning YOLOv8n-cls on an 84-class Indian traffic sign dataset with automated data splitting and augmentation.
- An interactive Streamlit web application supporting both image and video inference with annotated outputs.
- A comprehensive evaluation pipeline producing per-class accuracy, confusion matrices, and Top-1/Top-5 metrics.

2 Dataset

2.1 Source

We use the Indian Traffic Signs Prediction (85 Classes) dataset from Kaggle, authored by Sarang Dilip Jodh. The dataset contains real-world photographs of Indian traffic signs scraped from Indian roads, covering mandatory signs (blue circles), cautionary signs (red triangles), and prohibitory signs (red circles).

2.2 Data Preparation & Statistics

Our training pipeline (train.py) downloads the dataset, recursively discovers class folders containing image files, and filters out classes with fewer than 5 images. After filtering, 84 classes remain out of the original 85. The images are then randomly partitioned into 75% train, 15% validation, and 10% test splits while maintaining class-stratified structure.

Split	Classes	Images
Train	84	4,322
Validation	84	793
Test	84	532

Total	84	5,647

Table 1: Dataset statistics after filtering and splitting.

2.3 Class Imbalance

The dataset exhibits significant class imbalance. The largest class (NO_STOPPING_OR_STANDING) contains over 200 training images, while the smallest retained classes have around 5 images each. This long-tailed distribution is typical of real-world sign datasets, where common signs are photographed far more frequently than rare ones.

3 Methodology

3.1 Model Architecture

We employ YOLOv8n-cls, the classification variant of the YOLOv8 Nano architecture from Ultralytics. It uses a CSPDarknet53-based backbone with depthwise separable convolutions, followed by global average pooling and a fully connected layer mapping to 84 output classes. With roughly 3 million parameters and ImageNet-1K pre-trained weights, the model offers an excellent trade-off between accuracy and inference speed.

3.2 Training Configuration

Parameter	Value
Base model	yolov8n-cls.pt
Input resolution	320 x 320
Optimizer	AdamW
Learning rate (lr0)	1e-3
Weight decay	5e-4
Batch size	32
Epochs	30
Early stopping patience	15
Augmentation	Enabled (built-in)
Device	CPU

Table 2: Training hyperparameters.

3.3 Data Augmentation

YOLOv8's built-in augmentation pipeline is enabled during training. This includes random horizontal flipping, random scaling and translation, HSV colour-space jittering, mosaic augmentation (combining four training images), random erasing (40%), and RandAugment. These augmentations help mitigate overfitting on smaller, imbalanced classes and improve generalisation.

3.4 Loss Function

The model is trained with cross-entropy loss, the standard objective for multi-class classification: L = -sum(yi * log(yi_hat)) where C = 84 is the number of classes, yi is the one-hot ground truth, and yi_hat is the predicted probability from the softmax output.

4 Results

4.1 Overall Accuracy

We evaluate the trained model on the held-out test set (532 images across 84 classes) using evaluate.py, which computes Top-1 and Top-5 accuracy alongside per-class metrics.

Metric	Value

Top-1 Accuracy	93.98%
Top-5 Accuracy	99.62%
Number of classes	84
Test images	532

Table 3: Overall classification accuracy on the test set.

The Top-1 accuracy of 93.98% shows that the model correctly identifies the traffic sign class in the overwhelming majority of cases. The Top-5 accuracy of 99.62% means the correct class is almost always within the model's top five predictions, which is particularly useful for the interactive application where multiple candidate predictions are displayed.

4.2 Training Curves

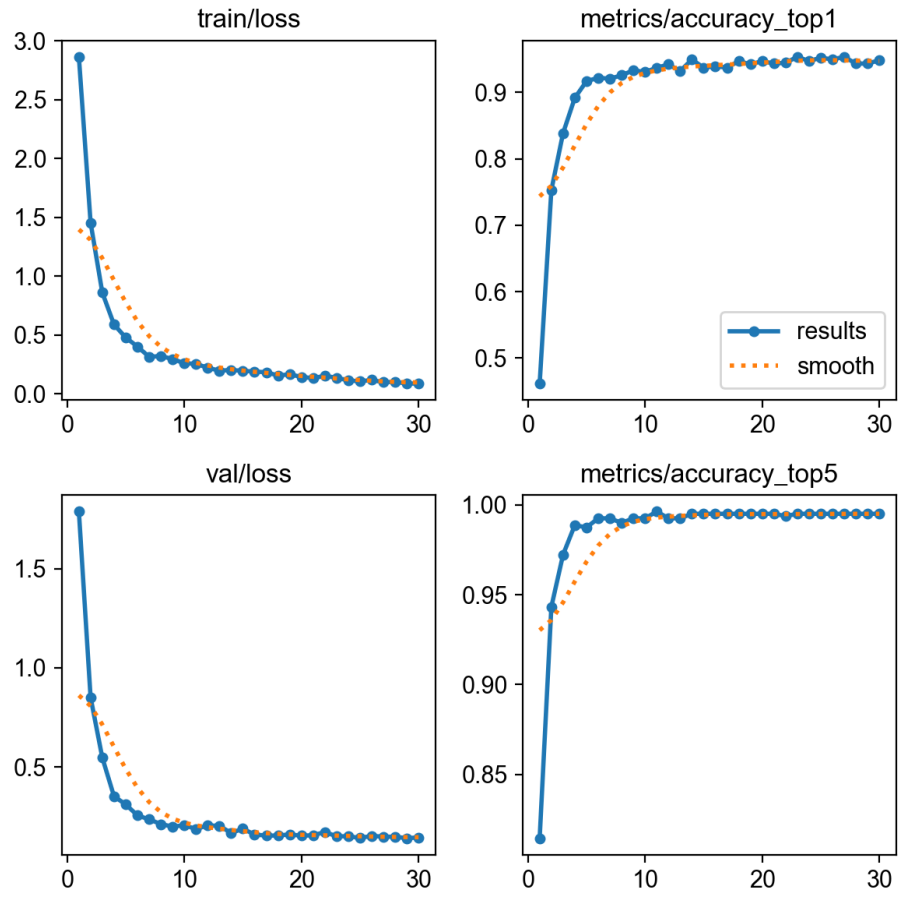

Figure 1: Training curves showing train loss, val loss, Top-1 and Top-5 accuracy over 30 epochs.

Figure 1 shows the training dynamics. The training loss drops sharply in the first 5 epochs and continues to decrease steadily, reaching roughly 0.09 by epoch 30. Validation loss stabilises around 0.14 after epoch 20. Top-1 validation accuracy climbs rapidly to about 89% by epoch 4 and gradually improves to a peak of 95.3% by epoch 23. Top-5 accuracy saturates above 99% by epoch 6. No early stopping was triggered, indicating the model continued to improve throughout all 30 epochs.

4.3 Per-Class Accuracy

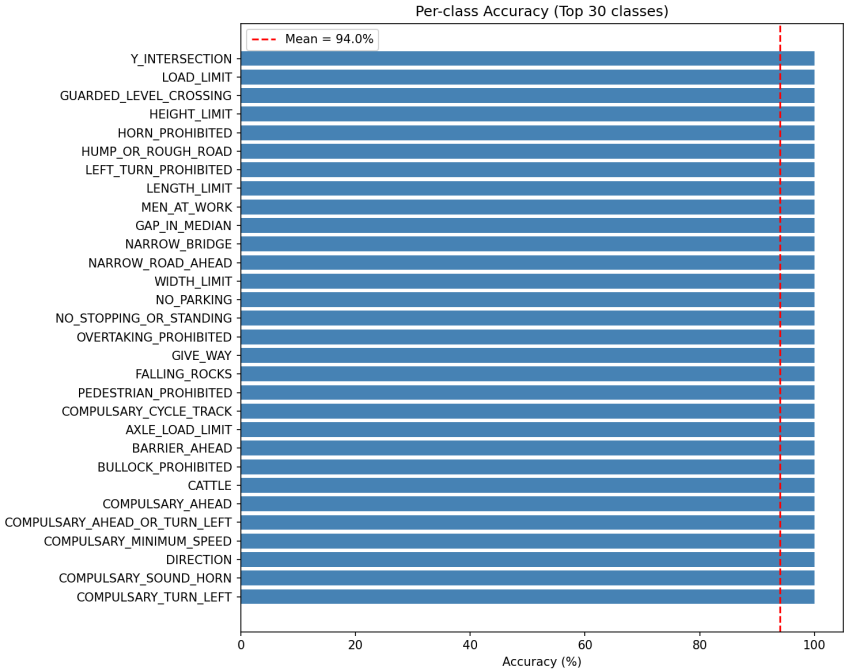

Figure 2: Per-class accuracy for the top 30 classes. The red dashed line indicates the overall mean accuracy of 94.0%.

Most high-frequency classes achieve 100% accuracy. Notable exceptions include some rare classes with limited training samples that show lower accuracy. Speed limit signs with similar appearances (e.g., SPEED_LIMIT_30 at 93.5%, SPEED_LIMIT_80 at 92.0%) show minor confusion due to visual similarity in the numeric digits.

4.4 Confusion Matrix

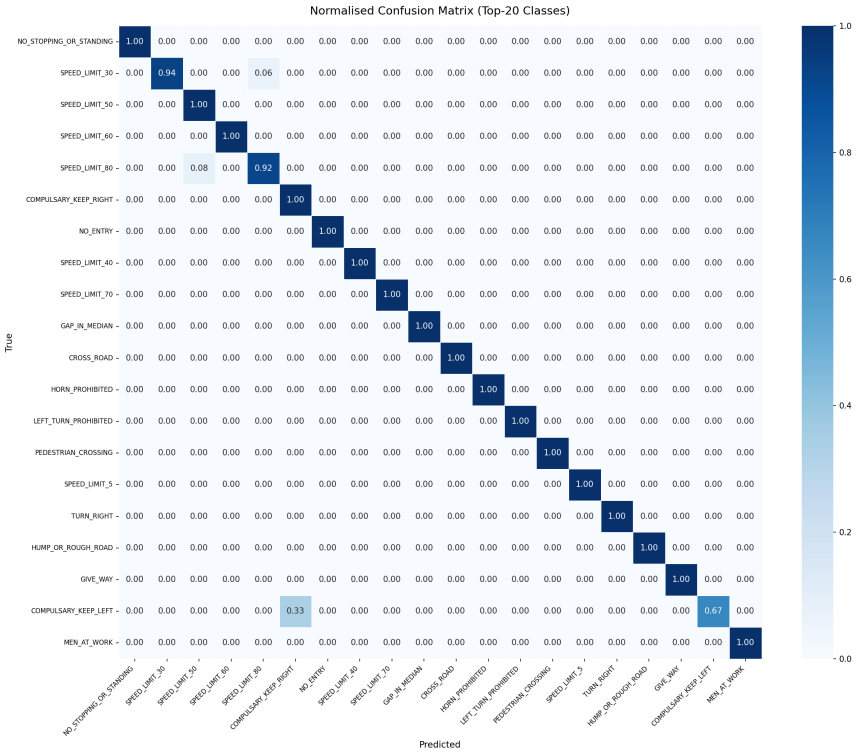

Figure 3: Normalised confusion matrix for the top 20 classes by frequency.

The confusion matrix shows strong diagonal dominance, confirming the model distinguishes well between common sign types. The most notable confusion occurs between SPEED_LIMIT_30 and SPEED_LIMIT_80 (6% misclassification), which is expected given both signs are circular red-border signs differing only in the numeric digit. COMPULSARY_KEEP_LEFT shows 33% confusion with COMPULSARY_KEEP_RIGHT, another understandable error given the near-identical sign designs.

4.5 Error Analysis

The primary sources of misclassification are:

- Visually similar classes: Speed limit signs differing only in the digit (e.g., 30 vs. 80) are occasionally confused.
- Rare classes: Classes with fewer than 10 training examples have insufficient data for reliable learning (10 classes scored 0% accuracy).
- Occlusion and weathering: Real-world images with partial occlusion, fading paint, or unusual viewing angles degrade recognition.

5 Application Demo

We deploy the trained model through an interactive Streamlit web application (app.py) that provides two inference modes: image classification and video analysis. The sidebar provides configurable settings including model weights path, confidence threshold (0.1-1.0), top-K predictions (1-5), and maximum video frames to process (10-500).

5.1 Image Classification Mode

Users upload a single photograph of a traffic sign. The app runs YOLOv8n-cls inference, annotates the image with a semi-transparent overlay showing top-K predictions and confidence scores, and displays a summary table with ranked predictions.

Figure 4: Streamlit app - Landing page with sidebar settings and file upload interface.

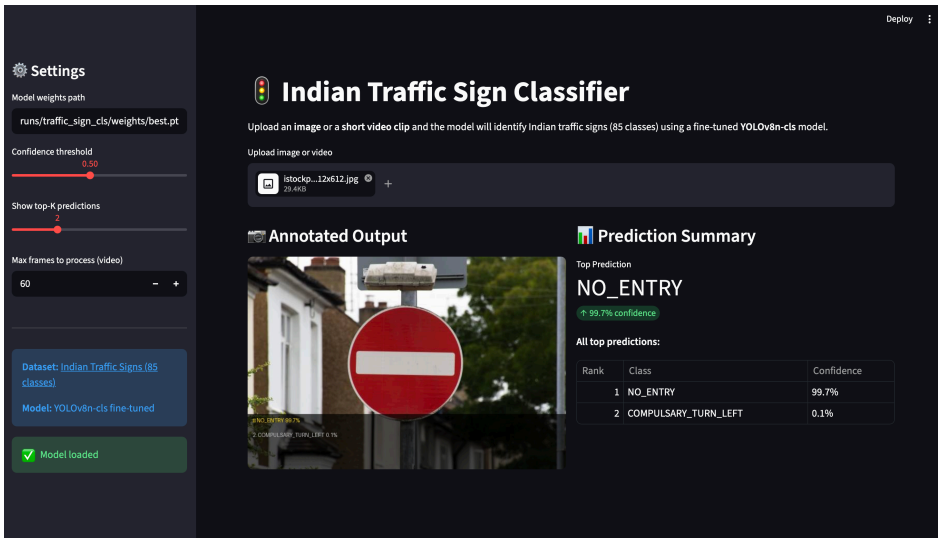

Figure 5: Image classification - NO_ENTRY sign correctly identified with 99.7% confidence.

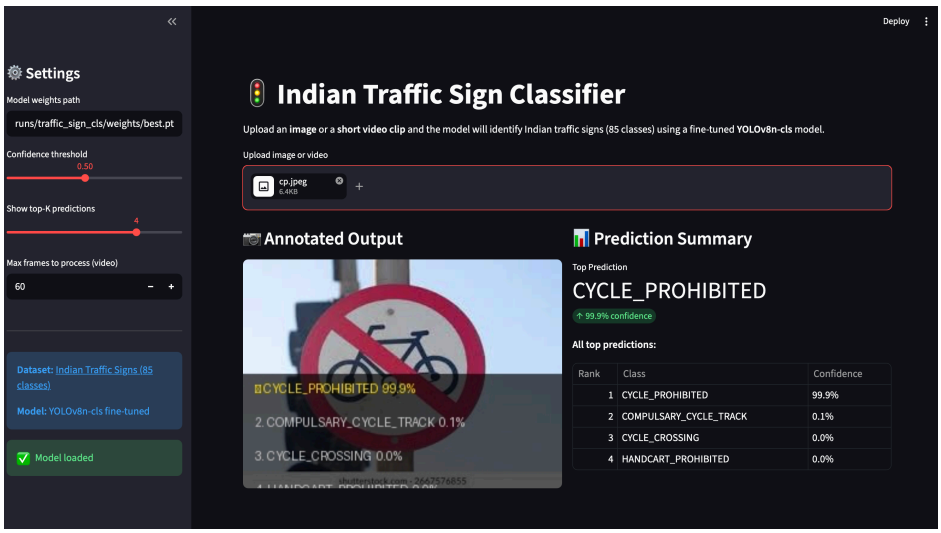

Figure 6: Image classification - CYCLE_PROHIBITED sign identified with 99.9% confidence, showing top-4 predictions.

5.2 Video Analysis Mode

Users upload short video clips (MP4, AVI, MOV). The application evenly samples up to a configurable number of frames, runs per-frame inference, and displays annotated sample frames alongside a detection summary with sign frequency table.

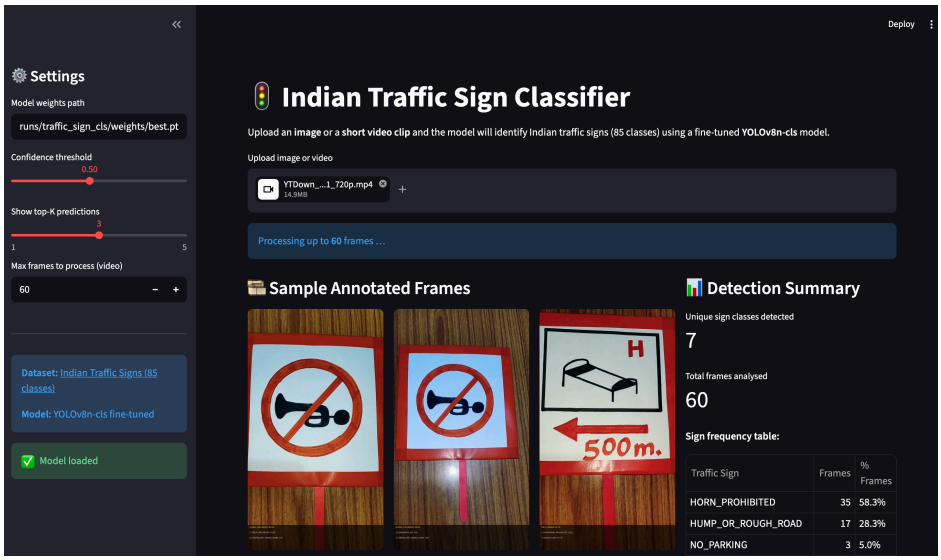

Figure 7: Video analysis mode - Processing a traffic signs video, detecting 7 unique sign classes across 60 frames.

6 System Architecture

File	Responsibility
train.py	Data download, ImageNet-style splitting, YOLOv8 fine-tuning, ONNX export
evaluate.py	Test-set evaluation, confusion matrix, per-class accuracy CSV and charts
app.py	Streamlit inference app (image + video modes)
requirements.txt	Pinned dependencies

Table 4: Project structure and file responsibilities.

Key design decisions:

- Cached model loading: @st.cache_resource ensures the YOLO model is loaded only once across Streamlit reruns.
- Frame sampling: np.linspace uniformly samples frames rather than processing every frame, enabling efficient video analysis.

- Annotation overlay: A semi-transparent RGBA overlay is composited onto images using PIL for readable predictions.

7 Limitations & Future Work

Current limitations:

- Class imbalance: With only 2-7 images for the rarest classes, the model struggles with rare sign types. Focal loss could help.
- Classification only: The current pipeline classifies pre-cropped sign images. A detection stage would be needed for real ADAS use.
- Domain gap: Training images may not represent the full variety of Indian road conditions (night-time, rain, fog).
- No temporal modelling: Video mode processes frames independently; temporal consistency is not exploited.

Future directions:

- Integrate a YOLOv8 detection head for end-to-end sign localisation + classification from uncropped dashcam footage.
- Apply focal loss or class-weighted loss to address the long-tailed class distribution.
- Deploy on HuggingFace Spaces for public access.
- Add object tracking (e.g., ByteTrack) for temporally consistent video predictions.

8 Acknowledgements

We thank the SMAI teaching team at IIIT Hyderabad for designing this assignment. We acknowledge the use of Claude (Anthropic) and Gemini (Google) for code scaffolding, debugging assistance, and report drafting. We also acknowledge Ultralytics YOLOv8 for the pre-trained model and training framework, and Kaggle for hosting the Indian Traffic Signs dataset. All evaluation, analysis, and experimental decisions are our own.

References

[1] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. https://github.com/ultralytics/ultralytics.

[2] J. Stallkamp et al., "Man vs. computer: Benchmarking ML algorithms for traffic sign recognition," Neural Networks, 2012.

[3] Ministry of Road Transport & Highways, "Road Accidents in India - 2022," 2023.

[4] S. Jodh, "Indian Traffic Signs Prediction (85 Classes)," Kaggle, 2022.

[5] J. Redmon et al., "You only look once: Unified, real-time object detection," Proc. IEEE CVPR, 2016.

[6] T.-Y. Lin et al., "Focal loss for dense object detection," Proc. IEEE ICCV, 2017.